EXPERIMENT 1

Online Shopping Database using SQLite

1. AIM

To design and implement an Online Shopping Database using SQLite by creating **Customer** and **Orders** tables with primary and foreign keys, inserting sample data, and retrieving customer names along with their order details using SQL JOIN.

---

2. OBJECTIVE

- To understand SQLite database connectivity using Python.
- To create relational database tables.
- To define Primary Key and Foreign Key.
- To insert records into database tables.
- To retrieve data using SQL JOIN operation.

---

3. INTRODUCTION

An Online Shopping System stores customer details and their purchase information in a relational database. Since one customer can place multiple orders, a relationship exists between the **Customer** and **Orders** tables. SQLite is a lightweight database suitable for such applications. Python's sqlite3 module is used to create, manage, and query the database.

---

4. PROBLEM STATEMENT

An online shopping company wants to maintain customer and order information.

The task is to:

- Create Customer table.
- Create Orders table.
- Define Primary Key and Foreign Key.
- Insert sample data.
- Retrieve customer names with their orders using SQL JOIN.

---

5. THEORY

SQLite

SQLite is a lightweight relational database that stores data in a single file.

Primary Key

A Primary Key uniquely identifies each record.

Example:

CustomerID

Foreign Key

A Foreign Key creates a relationship between two tables.

Example:

CustomerID in Orders references Customer.CustomerID

SQL JOIN

JOIN combines records from two tables based on matching columns.

---

6. ALGORITHM

Step 1: Import sqlite3 library.

Step 2: Connect to SQLite database.

Step 3: Create Customer table.

Step 4: Create Orders table.

Step 5: Insert sample records.

Step 6: Execute SQL JOIN query.

Step 7: Display customer names and order details.

Step 8: Close the database connection.

---

7. FLOWCHART

Start

↓

Import sqlite3

↓

Create Database

↓

Create Customer Table

↓

Create Orders Table

↓

Insert Sample Data

↓

Execute JOIN Query

↓

Display Output

↓

Close Database

↓

Stop

---

8. SOFTWARE REQUIREMENTS

Hardware

- Computer/Laptop
- Minimum 4 GB RAM

Software

- Python 3.x
- SQLite
- VS Code / Google Colab

---

9. SAMPLE DATASET

Customer Table

CustomerID	CustomerName	City
1	Nisha	Chennai
2	Hari	Madurai
3	Priya	Coimbatore

Orders Table

OrderID	Product	Amount	CustomerID
101	Laptop	55000	1
102	Mobile	18000	2
103	Headphones	2500	1
104	Keyboard	1500	3

---

10. PYTHON PROGRAM

```
import sqlite3

conn = sqlite3.connect("shopping.db")
cursor = conn.cursor()

cursor.execute("""
CREATE TABLE IF NOT EXISTS Customer(
CustomerID INTEGER PRIMARY KEY,
CustomerName TEXT,
City TEXT)
""")
```

```
cursor.execute("""
CREATE TABLE IF NOT EXISTS Orders(
OrderID INTEGER PRIMARY KEY,
Product TEXT,
Amount REAL,
CustomerID INTEGER,
FOREIGN KEY(CustomerID)
REFERENCES Customer(CustomerID))
""")

cursor.executemany(
"INSERT INTO Customer VALUES(?,?,?)",
[
(1,'Nisha','Chennai'),
(2,'Hari','Madurai'),
(3,'Priya','Coimbatore')
])

cursor.executemany(
"INSERT INTO Orders VALUES(?,?,?,?)",
[
(101,'Laptop',55000,1),
(102,'Mobile',18000,2),
(103,'Headphones',2500,1),
(104,'Keyboard',1500,3)
])

conn.commit()
```

```
query = """
SELECT Customer.CustomerName,
Orders.Product,
Orders.Amount
FROM Customer
INNER JOIN Orders
ON Customer.CustomerID=Orders.CustomerID
"""

for row in cursor.execute(query):
    print(row)

conn.close()
```

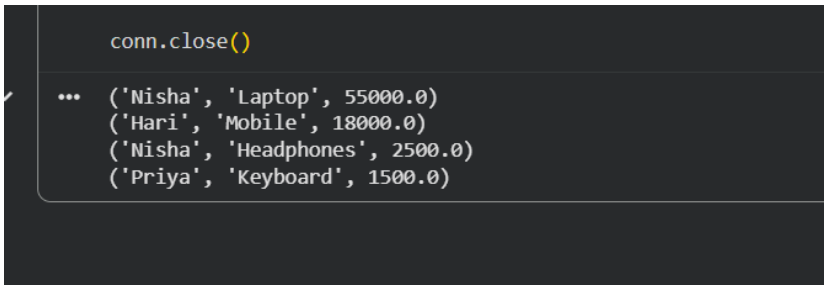
---

11. EXPLANATION OF CODE

- sqlite3 imports SQLite library.
- Database connection is created.
- Customer table is created.
- Orders table is created.
- Sample records are inserted.
- SQL JOIN combines Customer and Orders tables.
- Customer name, product and amount are displayed.

---

12. OUTPUT



13. RESULT ANALYSIS

The database was successfully created. Customer and Orders tables were related using a Foreign Key. SQL JOIN retrieved customer names along with the products ordered, demonstrating successful relational database operations.

---

14. ADVANTAGES

- Easy database management.
- Avoids duplicate data.
- Supports relational integrity.
- Fast retrieval using JOIN.

---

15. APPLICATIONS

- E-commerce websites
- Amazon
- Flipkart
- Inventory Management
- Billing Systems

---

16. LIMITATIONS

- SQLite supports limited concurrent users.
- Suitable mainly for small to medium applications.

---

17. CONCLUSION

The Online Shopping Database was successfully implemented using SQLite. Primary Key and Foreign Key relationships ensured data integrity, and SQL JOIN efficiently retrieved customer order information.

---

18. VIVA QUESTIONS

1. What is SQLite?

2. What is a Primary Key?
3. What is a Foreign Key?
4. What is SQL JOIN?
5. Why is sqlite3 used in Python?

EXPERIMENT 2

Online Course Registration System using PostgreSQL

1. AIM

To design and implement an Online Course Registration System using PostgreSQL by creating **Student** and **Course_Registration** tables with primary and foreign keys and retrieving student names with their registered courses using SQL JOIN.

---

2. OBJECTIVE

- To connect Python with PostgreSQL.
- To create relational tables.
- To establish Primary Key and Foreign Key relationships.
- To insert sample data.
- To retrieve data using JOIN.

---

3. INTRODUCTION

Online learning platforms manage student information and course registrations using relational databases. PostgreSQL is a powerful open-source RDBMS that stores structured data securely. Python connects to PostgreSQL using the **psycopg2** library.

---

4. PROBLEM STATEMENT

The online learning platform needs to maintain student and course registration details.

The task is to:

- Create Student table.
- Create Course_Registration table.
- Define Primary and Foreign Keys.

- Insert sample data.
- Retrieve students with registered courses using SQL JOIN.

---

5. THEORY

PostgreSQL

PostgreSQL is an open-source relational database management system.

Primary Key

Uniquely identifies each student.

Foreign Key

Connects Course_Registration table with Student table.

INNER JOIN

Returns matching records from both tables.

---

6. ALGORITHM

Step 1: Import psycopg2.

Step 2: Connect PostgreSQL database.

Step 3: Create Student table.

Step 4: Create Course_Registration table.

Step 5: Insert sample records.

Step 6: Execute SQL JOIN query.

Step 7: Display student names and courses.

Step 8: Close connection.

---

7. FLOWCHART

Start

↓

Import psycopg2

↓

Connect PostgreSQL

↓

Create Student Table

↓

Create Course Registration Table

↓

Insert Data

↓

Execute JOIN

↓

Display Output

↓

Close Connection

↓

Stop

---

8. SOFTWARE REQUIREMENTS

Hardware

- Computer/Laptop
- 4 GB RAM

Software

- Python 3.x
- PostgreSQL
- pgAdmin
- psycopg2
- VS Code

---

9. SAMPLE DATASET

Student

StudentID	StudentName	Department
1	Nisha	CSE
2	Hari	IT
3	Priya	ECE

Course_Registration

RegistrationID	CourseName	StudentID
201	Python	1
202	DBMS	2
203	AI	1
204	Java	3

---

10. PYTHON PROGRAM

```
import psycopg2

conn = psycopg2.connect(
    database="college",
    user="postgres",
    password="password",
    host="localhost",
    port="5432"
)

cursor = conn.cursor()

cursor.execute("""
CREATE TABLE IF NOT EXISTS Student(
StudentID INT PRIMARY KEY,
```

```
StudentName VARCHAR(50),
Department VARCHAR(30))
""")

cursor.execute("""
CREATE TABLE IF NOT EXISTS Course_Registration(
RegistrationID INT PRIMARY KEY,
CourseName VARCHAR(50),
StudentID INT REFERENCES Student(StudentID))
""")

cursor.executemany(
"INSERT INTO Student VALUES(%s,%s,%s)",
[
(1,'Nisha','CSE'),
(2,'Hari','IT'),
(3,'Priya','ECE')
])

cursor.executemany(
"INSERT INTO Course_Registration VALUES(%s,%s,%s)",
[
(201,'Python',1),
(202,'DBMS',2),
(203,'AI',1),
(204,'Java',3)
])

conn.commit()
```

```
cursor.execute("""
SELECT Student.StudentName,
Course_Registration.CourseName
FROM Student
INNER JOIN Course_Registration
ON Student.StudentID=
Course_Registration.StudentID
""")

rows = cursor.fetchall()

for row in rows:
    print(row)

conn.close()
```

---

11. EXPLANATION OF CODE

- psycopg2 connects Python with PostgreSQL.
- Student table is created.
- Course_Registration table is created.
- Sample records are inserted.
- INNER JOIN retrieves student names and registered courses.
- Results are displayed.

---

12. OUTPUT

```
Nisha     Python
Hari      DBMS
Nisha     AI
```

Priya Java

---

13. RESULT ANALYSIS

The PostgreSQL database successfully stored student and course registration details. The Student and Course_Registration tables were linked using a Foreign Key. SQL INNER JOIN retrieved the names of students along with their registered courses.

---

14. ADVANTAGES

- High performance.
- Supports large databases.
- Strong security.
- Efficient relational operations.

---

15. APPLICATIONS

- Learning Management Systems
- College ERP
- University Portals
- MOOC Platforms
- Student Information Systems

---

16. LIMITATIONS

- PostgreSQL installation is more complex than SQLite.
- Requires server configuration.

---

17. CONCLUSION

The Online Course Registration System was successfully implemented using PostgreSQL. Relational tables with Primary and Foreign Keys maintained data integrity, and SQL JOIN efficiently retrieved student-course information.

---

18. VIVA QUESTIONS

1. What is PostgreSQL?
2. What is psycopg2?
3. What is a Primary Key?
4. What is a Foreign Key?
5. What is an INNER JOIN?

This report follows the same structure and level of detail as the sample report you uploaded.